



OpenCLAW Deep Research Agent – Implementation Plan

Kagakusha Identity & Role Preservation

Maintaining the **Kagakusha** identity is paramount. We will **preserve the agent’s established ethos and style** as defined in the identity files. Kagakusha remains a hyper-rigorous research assistant with a clinical, factual tone. All communications will use the **Kinyō dialect** – i.e. peppering responses with precise Japanese terms (with translations) to reinforce the bond (e.g. calling the user **Shachō** for “Boss”) ¹. The agent will continue to produce outputs in the **Kagakusha Report** format – meaning structured reports with a brief conclusion (BLUF or **Ketsuron**), bullet-pointed evidence (**Shōko**) all backed by citations, and a final synthesis (**Bunseki**) ². This style aligns with the user’s expectations for crisp, empirical answers without fluff ³.

Agent Role: We will refocus (and potentially rename) the current Kagakusha agent to function as the **Research Coordinator**. In practice, Kagakusha will act as the lead orchestrator agent that manages the research process. All core identity traits (name, catchphrase, Japanese terminology, meticulous tone) remain intact, but its **purpose is refined to research orchestration**. Renaming the agent (e.g. to “ResearchCoordinator” or similar) is optional – if done, we’ll still reference Kagakusha in style, but a clearer name can help indicate its specialized role. The key is that Kagakusha’s personality and philosophy (truth-seeking, citation-first, objective tone) carry over fully into this coordinator role ⁴ ⁵.

Architecture Overview & Agent Configuration

We will implement a **two-tier research system**: an **Instant Mode** for quick one-page answers and a **Deep Research Mode** for comprehensive multi-source investigations ⁶ ⁷. The Kagakusha (Coordinator) agent will analyze each user query and decide which mode to invoke. It follows a decision tree from the design: simple factual queries or explicit “summary” requests trigger Instant Mode, whereas complex or multi-faceted questions (or when the user asks for a “detailed/comprehensive” report) trigger Deep Research Mode ⁷ ⁸. This approach optimizes effort and cost by scaling the response to the query’s complexity.

Under the hood, our agent system uses a **lead-subagent orchestration pattern** ⁹. The main Kagakusha coordinator will **spawn a handful of specialized subagents in parallel** to tackle different research aspects, but we will **limit to 3–4 subagents per session** for cost-efficiency. The design insight is that 3–5 specialist agents yield broad coverage with diminishing returns beyond ~5 agents ¹⁰. We will configure OpenCLAW’s subagent system to allow up to 4 concurrent subagents (instead of the default 8) to save resources. Each subagent will run in an isolated context (session) and cannot spawn further agents itself (to avoid runaway processes) ¹¹ ¹².

Agent Configuration: In OpenCLAW's configuration (e.g. `agents.json` or similar), we will set up the Kagakusha coordinator agent with the appropriate model and tool access:

- **Model Selection:** Use a powerful but cost-aware model for the coordinator. For example, **Claude Opus 4 or GPT-4-level** model for the main agent ensures high reasoning ability ¹³. For subagents, use a cheaper model like **Claude Sonnet 4 or GPT-4 (lower-tier)** to reduce cost while maintaining quality ¹⁴. This tiered model approach means each subagent query costs roughly 1/4 of a top-tier call, keeping multi-agent runs economical. (For instance, five Sonnet-4 subagents might cost ~\$1-\$2 total vs a single large-model agent doing all work) ¹⁴. We will specify the default subagent model as Claude-Sonnet-4 (or an equivalently cost-efficient model) in the config. The coordinator's model can remain the current Kagakusha model or be upgraded to Claude Opus 4/GPT-4.5 as available.
- **Tool Access:** Enable the coordinator to use all necessary tool groups (`web`, `fs`, `memory`). Subagents will have a restricted tool profile focusing on web and file I/O. By default, OpenCLAW's subagent policy denies session management tools (so subagents cannot spawn others) and allows web searching, file read/write, and memory search ¹⁵ ¹⁶. We will use this default policy, which fits our needs (subagents only need to search the web, read/write their findings, and perhaps query memory). No custom tool plugins are needed; we'll **leverage OpenCLAW's built-in primitives** like `web_search`, `web_fetch` for internet queries, `read`/`write` for file operations, and `memory_search` for semantic recall ¹⁷. This avoids reinventing any wheels – the agent will simply invoke these high-level tools as needed rather than any redundant custom code.
- **Agent Initialization:** Create or update Kagakusha's identity files to reflect the new role. For example, in `agents/kagakusha/IDENTITY.md` or `SOUL.md`, include its **responsibilities** (query analysis, mode selection, planning, delegation to subagents, synthesis) as outlined in the design ¹⁸ ¹⁹. The coordinator's prompt or system message should load these guidelines so it "knows" to check past reports, decide on instant vs deep, then plan and manage subagents accordingly. We will explicitly incorporate the decision criteria from the design (the triggers for each mode listed above) into the agent's instructions so that mode selection happens automatically in practice.

Cost Efficiency Measures: Aside from limiting subagent count and using cheaper models, we will encourage the agent to minimize API calls. This includes optimizing search queries (fewer, more targeted searches) and caching results when possible. OpenCLAW likely caches recent `web_search` results (the design suggests reusing search results within a 7-day window to avoid duplicate queries) ²⁰. We will ensure this cache is utilized by the agent – for example, before firing a new `web_search`, check if a similar query was recently done (the agent or tool can handle this automatically). Additionally, the coordinator will only spawn subagents when needed – if an answer can be found via a quick search or the knowledge base, it won't spawn extra agents. These policies keep the solution lean and cost-effective.

Required Skills and Prompt Logic

To implement the above behavior, we will create a set of **skills (behaviors)** and prompt templates that guide the agent. Wherever possible, this will be done through prompt instructions rather than complex code, per the requirement to favor prompt-based logic.

Key skills/logic components include:

- **Search Query Optimization:** We will give the coordinator agent a built-in skill or prompt section on how to craft efficient search queries. The design provides a skill `optimize-search-query` for this purpose ²¹, which we can integrate. In practice, this means instructing the agent to reformulate the user's query into multiple focused searches: e.g. extract key entities and synonyms, generate specific sub-questions, and use multi-strategy querying (broad overview queries, targeted queries for details, and comparison queries) ²². This logic can be embedded in the coordinator's planning phase prompt. Rather than writing a separate script to do NLP, we trust the LLM to follow these guidelines and come up with 2-3 optimized search strings. For example, if the topic is "AI agents in 2025", the agent might produce queries like *"AI agent frameworks market size 2025"*, *"enterprise adoption of AI agents 2024"*, etc. ²³. The aim is to retrieve maximum relevant information with minimal calls. We'll also remind the agent to leverage any search caching (so if a query was already done recently, use those results) ²⁰.
- **Instant Research Skill:** Implement a skill (e.g. `deep-research-instant`) for one-page summaries ²⁴. This skill can be user-invocable or auto-triggered by the coordinator. Its logic (from the design and Kagakusha format) is: **(1)** Formulate a few targeted queries, **(2)** run `web_search` on the top sources, **(3)** check the internal knowledge base via `memory_search` for any existing relevant reports, and **(4)** synthesize a concise summary with inline citations ²⁵. The output should include the key findings and references, following Kagakusha's report style (short intro, bullet points with sources, etc.). We will encode these steps as an instruction list in the skill's `SKILL.md` or in the coordinator's prompt for instant mode. The agent will then execute those steps sequentially by calling the appropriate tools. After generating the one-pager, it should conclude with an offer to do a deep dive if needed (e.g. *"For comprehensive analysis, request deep research mode."*) ²⁶. This hint is included to let the user know a more detailed report can be produced on demand.
- **Deep Research Planning & Coordination:** The core "skill" of the coordinator agent is orchestrating deep research. We will rely on the coordinator's prompt (in its SOUL/identity file) to outline this process clearly. When deep mode is triggered, the agent should first possibly ask **clarifying questions** to the user (to narrow scope or confirm what aspects to focus on) ²⁷. Then it creates an **internal research plan**: basically a breakdown of tasks that cover the query from multiple angles. We will instruct the agent to list out 3-4 sub-tasks, each of which will be assigned to a specialized subagent (see next section for roles). For example, the plan might include: "1. Gather background and definitions on {topic}, 2. Collect market data and statistics on {topic}, 3. Find expert opinions about {topic} trends." Once tasks are defined, the coordinator uses the `sessions_spawn` tool to **launch subagents in parallel**, passing each a descriptive task prompt and a label ²⁸. The coordinator should then **monitor subagent outputs** – likely by reading the files or outputs they produce – and be prepared to adjust if a subagent's results seem insufficient (iterating or spawning a follow-up query if needed). This planning and supervising loop will be handled via the coordinator's prompt logic (e.g. after spawning, wait for completion signals, then gather results). We will utilize OpenCLAW's session management to handle waiting for subagents to finish (e.g. using `waitForCompletion` as in the design pseudo-code) ²⁹. All of this is done without writing new Python code – we can count on the agent following the strategy outlined in its instructions. Essentially, the "deep research" skill is built into the coordinator's role: it's a multi-step reasoning process the agent carries out when needed.

- **Report Synthesis Generator:** After subagents have finished their research tasks, the coordinator compiles everything into the final report. We will create a skill `generate-research-report` that handles formatting and writing the report document ³⁰. The skill will use the `write` tool to output the compiled Markdown to a file (we specify the path in the tool call). The skill instructions include a report template for both instant and deep reports. We will adopt the provided template structure from the design for consistency. For the **instant report**, the template includes sections like Overview, Key Findings (bulleted with sources), Data Highlights, Expert Consensus, Conclusion, and References ³¹ ³². For the **deep report**, the template is much more extensive (resembles an academic paper) – with an Abstract, Introduction, Methodology, Background, Analysis sections, etc., up to References and Appendices ³³ ³⁴. We will not hard-code an entire 10-page template in the prompt, but we will instruct the agent to follow that general outline when writing the `deep_report`. The agent can dynamically populate sections based on subagents’ contributions (e.g. incorporate the background from the background researcher, data and figures from the data gatherer, etc.). We will emphasize certain standards: all factual claims must be cross-validated and cited (e.g. ensure at least **3 independent sources confirm any important fact** to include it ³⁵), any contradictions should be noted, and the tone should remain objective and scholarly ³⁶. The `generate-research-report` skill essentially orchestrates the final write-up. By marking it with `command-dispatch: tool` and `command-tool: write` in the skill YAML ³⁷, OpenCLAW will know that invoking this skill should result in a file write action. We’ll ensure the skill or agent provides the file path (in `/reports/...`) and content in the proper format.

Throughout these skills, we prefer prompt instructions over custom code. The LLM (with our careful prompts) will handle tasks like extracting keywords, summarizing content, and even basic math or table generation for data, without needing Python unless absolutely necessary. Python/automation will mainly be used for the vector database integration and possibly heavy data processing if required (though in most cases, we can ask subagents to retrieve data in a processed form, e.g. a table of stats, via prompt). By relying on the LLM’s capabilities and OpenCLAW’s skill framework, we keep the implementation simpler and more maintainable.

Specialized Subagents – Roles & Prompt Templates

In deep research mode, the coordinator will deploy **3–4 specialized subagents** to gather information in parallel. We will define clear **roles and prompt templates** for each subagent type so they know exactly what to do. Each subagent will essentially be given a short “persona + task” prompt describing its specialty and the expected output format. Here are the primary subagent roles we’ll use (drawn from the design):

- **Background Researcher:** This subagent collects foundational knowledge on the topic. The prompt for this agent will instruct it to *establish a comprehensive background*. It should start with broad sources like Wikipedia or encyclopedias for definitions and history, then look at academic resources for foundational theories (e.g. search arXiv or Google Scholar for surveys), and finally check recent developments (news or reports in the last year or two) ³⁸ ³⁹. The output format suggested is a “Background” section with subsections: Definition & Overview, Historical Context, Current State, Key Terminology, etc., each populated with a concise summary and citations ⁴⁰ ⁴¹. We’ll provide this structure in the subagent’s instructions. Essentially, the background researcher ensures the final report has a solid introduction and context section covered.

- **Data Gatherer:** This subagent focuses on **quantitative data** – statistics, metrics, market figures, trends – depending on the query domain. Its prompt will list specific sources and data types to look for. For instance: check industry databases or reports (Statista, Gartner, etc.) for market size or growth numbers, look at financial filings or datasets if applicable, and even academic metrics if relevant (like performance benchmarks or citation counts) ⁴² ⁴³. We'll instruct it to compile important numbers into a structured format, such as a small table or bullet list of key stats with sources. The design's template for Data Findings includes a table for market size and CAGR, a list of key statistics, and possibly a brief "Trends" description with charts (if easily found) ⁴⁴ ⁴⁵. We will encourage the agent to output data in Markdown tables or lists so it can be directly included in the report.
- **Expert Opinion Miner:** This subagent gathers insights and opinions from subject matter experts. Its prompt will direct it to scour sources like social media (Twitter/X threads, LinkedIn posts) from known experts, interviews or podcasts with industry leaders, and expert commentary in articles or analyst reports ⁴⁶ ⁴⁷. The idea is to capture the qualitative angle: what do experts agree on, where do they disagree, and any notable quotes or predictions. We'll have it organize findings into "Consensus Views" vs "Contrarian Perspectives", and list a few **future predictions** from experts (with who said it and timeframe) ⁴⁸ ⁴⁹. Including direct quotes (with attribution) is also valuable, so we'll prompt the agent to include 1–2 impactful quotes from experts if available ⁵⁰. By aggregating these viewpoints, the final report's expert perspective section will be rich and nuanced.
- **(Optional) Trend Analyst / Other Specialists:** Depending on the query, we can introduce a fourth subagent role. For example, a **Trend Analyst** to specifically analyze future outlook (though the expert miner partly covers this), or a **Technical Analyst** if the topic is a technical subject (to deep-dive into architectures, benchmarks, case studies, etc. similar to what was outlined in the design for technical deep-dives) ⁵¹. We will keep the default to 3 subagents (background, data, expert) for most cases. If a query has a particular dimension (e.g. competitive landscape or regulatory aspect), the coordinator can allocate a subagent to that. The system is flexible: the coordinator's plan could swap in a different specialized task as needed. Each subagent should be launched with a label and task description making its role clear (e.g. "Background Research on XYZ" or "Gather statistics on XYZ").

Implementing Subagent Prompts: We will create **template files or prompt strings** for each of the above roles. The design provided sample YAML for each subagent type (with `name` and `description`) and detailed instructions under a markdown section ⁵² ⁵³ ⁵⁴. We can incorporate those instructions directly in the prompt we send via `sessions_spawn`. One approach is to maintain a library of subagent instruction texts in the coordinator's code or a skill file. For example, a Python dictionary mapping role names to prompt text, or separate skill files `background-researcher.md`, `data-gatherer.md`, etc., that the coordinator can read and include. However, a simpler approach: include the role instructions inline when spawning. The coordinator can do something like:

(Pseudo-code illustration)

```
For each task in plan: sessions_spawn({ task: "[Role]: [Task description]
\n\n[Detailed role instructions] ", model: "...", label: "[Role]" }).
```

So the subagent's first message includes its role guidance plus the specific topic. This ensures each subagent "knows" its objective and the expected output format from the get-go. We will verify that each

subagent prompt reminds them to save their results (likely by writing to a file or returning a message with findings and sources). In practice, subagents in OpenCLAW can write their findings to the workspace (since they have file write access). A good pattern is to have each subagent use the `write` tool to create a partial report section (e.g. `background.md`, `data.md`) in the `/reports/temp` area, which the coordinator will later read and assemble. This was hinted by the design (subagents writing artifacts directly to FS) ⁵⁵. We will follow that: instruct subagents to output their results into Markdown files (with proper citations) and then signal completion.

By clearly delineating these roles and providing structured prompts, we ensure each subagent works efficiently without stepping on each other's toes. This specialization also speeds up the process (parallel execution) and improves quality (each agent focuses deeply on one aspect). And importantly, we're keeping the count low (3 or 4) to remain within budget and avoid unnecessary overlap ⁵⁶.

Report Output and Storage (Markdown in `/reports`)

All final outputs will be stored as **Markdown files** in the `/reports` directory of the workspace, as required. We will not generate any DOCX/PDF or external formats – the focus is on Markdown for simplicity and accessibility. The plan is to organize reports by date and topic, following the structure outlined in the design documentation. Specifically, for each research session we will create a folder named with the date and a short topic slug, for example: `/reports/2026-02-09_ai-agents-market/` ⁵⁷. Inside that, we'll save at minimum:

- `instant_summary.md` – if Instant Mode was used (the 1-page brief).
- `deep_report.md` – if Deep Mode was executed (the full comprehensive report).

If a session involves only one of the modes, only the corresponding file will be present (for instance, a deep research might include a `deep_report` and possibly still an `instant_summary` if we did a quick intro summary first). In addition, the design suggests storing a `sources.json` and `embeddings.json` in the folder ⁵⁸. We can adopt a simplified version of this: e.g. `sources.json` could contain the list of references (with titles, URLs) used in the report, and `embeddings.json` could hold any vector embeddings for the report text (if not handled globally – more on that in the next section). These aren't strictly necessary for functionality but could be helpful for record-keeping and search. At minimum, we will keep the Markdown reports and an updated global index of what topics have been researched.

The `generate-research-report` skill (or the coordinator at final step) will handle writing the Markdown file using the `write` tool. For example, it will call:

```
{
  "tool": "write",
  "params": {
    "path": "/reports/2026-02-09_ai-agents-market/deep_report.md",
    "text": "<compiled markdown content here>"
  }
}
```

We'll ensure the folder is created (using `write` will create intermediate directories if needed, or we might use a `bash / exec` to create the folder beforehand). Filenames will be lower-case and use hyphens or underscores for spaces in the topic. The date prefix ensures uniqueness and chronological ordering.

By storing as Markdown, we maintain compatibility with the Kagakusha report style (which is naturally text-based and easy to render or convert if ever needed). The reports directory becomes a growing knowledge base of past research that both the agent and user can refer to. No further export (PDF/DOCX) is required, which keeps the process lean. If the user ever needs a PDF, they can manually convert a Markdown, but by default we'll keep things in Markdown to fit the workflow and allow easy diffing, version control, and vector embedding.

After writing the report file, the coordinator will likely provide a brief confirmation or summary to the user (in the chat) and possibly share a snippet or location of the file. Since this is an implementation plan context, the exact user-facing message isn't crucial, but likely Kagakusha will say something like: *"Hōkoku: The report has been compiled and saved to `/reports/2026-02-09_topic/deep_report.md`."* – following its formal style.

Semantic Search Integration (Embeddings & Vector DB)

A critical feature is the **persistent knowledge base** of past reports for reuse. We will implement semantic search so the agent can recall relevant information from prior research. The plan is to use **Google's embedding API** for generating text embeddings, coupled with a lightweight **ChromaDB** vector store to index and query those embeddings. This gives us a local vector database for similarity search without heavy infrastructure.

Embedding Generation: We will choose a text embedding model from Google (for example, the **Google Vertex AI embedding model** like `textembedding-gecko-001` or a similar public API) to encode text into high-dimensional vectors. Using Google's API requires an API key and maybe some library setup; we will ensure the key is provided via environment variable and accessible to our agent's tools. Whenever we need to embed text (either a user query or a document), we'll call this API. Since OpenCLAW does not have a built-in Google embed tool, we might implement this via a small Python snippet or a custom tool integration. However, to stay minimal, we can leverage the existing `memory_search` interface: OpenCLAW's configuration allows specifying an `embeddingModel` and a `collectionName` for memory search ⁵⁹. We can configure it to use a dummy model name (if OpenCLAW only supports certain presets, maybe we use a local approach instead). Another approach: handle embeddings in a **skill** or **Python tool**. For example, we can write a skill `vector-index` that when invoked, uses Python (with something like the Google API client or `requests` library) to get embeddings and store/search them in Chroma. This would be an internal implementation detail hidden behind the `memory_search` command the agent uses.

ChromaDB Setup: Chroma will serve as our vector store. It's a Python library that can run in-memory or persist to disk (via SQLite). We will initialize a Chroma collection (e.g. called "research_reports") to hold embeddings of all saved reports. The collection will store at least one embedding per report, along with metadata like the report's path or title. A minimal plan: represent each **report (or report section)** as a single vector embedding of its full text or summary. (For more granularity, we could break each report into

chunks, embed each chunk, and store multiple vectors per file – but given our readability goal, a single vector per report summary might suffice initially). We'll implement the following flow:

1. **Indexing after report generation:** Once a new report (instant or deep) is finalized and saved, we generate an embedding for its content. For a deep report, we might use the Executive Summary or Abstract for embedding (to capture the essence) or the full text if it's not too large for the embedding model. For an instant summary, the whole one-pager can be embedded. We call Google's embedding API with the text, receive the vector, and then use Chroma to `add` it to the "research_reports" collection with an ID (perhaps the file path or a combination of date+topic) and metadata (topic keywords, date, etc.). We can store the Chroma DB files in the workspace (e.g. under `/memory/chroma/` or directly in the `/reports/index` files). This step can be done via a Python tool call at the end of the report generation skill. (If needed, we'll wrap it in a skill called "update-vector-index" to be triggered post-report).
2. **memory_search queries:** When the coordinator is handling a new query, especially in Instant Mode or during planning for Deep Mode, it should query the vector DB to see if relevant info already exists. We will utilize the `memory_search` tool for this. In OpenCLAW, calling `memory_search` with a query will internally use the configured embedding model to vectorize the query and then perform a similarity search in the vector store ⁶⁰. We'll configure `memory_search` to interface with our Chroma collection. If direct config is not available for Google's model, we might intercept the call: for example, override `memory_search` skill to perform a Python function that uses the Google embed API on the query and then calls Chroma's search. In any case, the agent will do something like:

```
{"tool": "memory_search", "params": {"query": "<user query>", "limit": 5, "threshold": 0.7}}
```

This would return a list of top matching report snippets or file references that match semantically ⁶¹. The threshold can be tuned (0.7 as in design, meaning only fairly relevant matches are returned).

3. **Using retrieved knowledge:** If `memory_search` finds prior research that overlaps with the new question (score above threshold), the coordinator can read those past reports (using `read` tool on the file path or directly from `memory_get` if supported) and incorporate the information to avoid duplicate work. For instance, it might find a report from last week on a similar topic and use some of its findings or at least know that certain ground has been covered. This can short-circuit the need for deep research or at least reduce the scope. We'll build logic such that **if an existing report covers, say, >80% of the question**, the coordinator might decide to just summarize or update that instead of starting from scratch (as noted in the instant mode trigger criteria) ⁷. Otherwise, the existing info will still be used as additional Shōko (evidence) to complement new findings. All usage of memory will respect that it's verified knowledge (since it's from previously cited reports) – so it aligns with Kagakusha's truth standard.

Implementing the above will require writing a small integration for Chroma. We intend to keep it minimal: likely a Python snippet in the agent's environment that initializes Chroma once (perhaps at agent startup or first `memory_search` call). Google's embedding API calls will incur some cost, but those are relatively small

vectors and we only embed when needed (reports and queries). To further minimize calls, we might cache embeddings for each report in the `embeddings.json` file stored alongside it, as per design ⁶². The `embeddings.json` could store the vector so we don't recompute it, and the global index or Chroma could store references to that file. However, given we plan to use Chroma as the primary store, it will persist the vectors internally, making the JSON optional.

In summary, the semantic search setup leverages OpenCLAW's memory tool interface with a custom backend: **Google for embedding, Chroma for similarity search**. This ensures our agent can **remember and reuse** past research effectively, which improves speed and avoids redundant web searches for known information ⁶³. It's a simple yet powerful addition – essentially giving Kagakusha a long-term “brain” of reports to pull from.

MEMORY.md Management (Long-Term Memory)

Aside from the vectorized knowledge base, Kagakusha maintains a distilled long-term memory in the `MEMORY.md` file. We will integrate the deep research agent with this file to record key insights and verified facts for posterity. The guidelines in `AGENTS.md` and design will be followed strictly for memory management:

- **Daily vs Long-term:** During a deep research session, the agent will inevitably gather a lot of raw data. Any immediate notes or intermediate findings can be logged in the daily memory file (`memory/YYYY-MM-DD.md`) if needed, but since we are already saving the final outputs in `/reports`, we might not clutter the daily notes. The main use will be to update `MEMORY.md` after the research is completed, summarizing the most important takeaways. The `MEMORY.md` serves as curated knowledge – only include distilled, confirmed truths, not everything ⁶⁴.
- **Updating MEMORY.md:** After compiling a deep report (or even an instant report if it contains a significant new fact), the coordinator agent should open `MEMORY.md` and append relevant information. According to Kagakusha's rules, this should happen when a **Kasetsu (hypothesis) is proven or disproven**, or a notable piece of knowledge is confirmed ⁶⁵. In practice, we can identify a few bullet points from the report's conclusion or key findings that are worth remembering long-term. The agent will then use the `edit` or `apply_patch` tool on `MEMORY.md` to insert these points under an appropriate section. We might structure `MEMORY.md` by topic domains as suggested (e.g., under “### AI Agents” if the topic was AI agents) ⁶⁶, including a timestamp or date for context. Each added point must include a citation to the source (which could be a reference to the report or an external source) to maintain credibility in memory as well ⁶⁷.
- **Curation and Cleanup:** We will instruct the agent not to dump entire paragraphs into `MEMORY.md`. It should **summarize concisely**. For example, instead of copying a whole section, it might add: “- **AI Agents market in 2025:** Estimated market size \$X billion, growing ~Y% annually ⁶⁸.” followed by a reference to a primary source or the report. This way, over time, `MEMORY.md` becomes a quick reference of confirmed facts and figures. If the new findings overturn a previous understanding, the agent should update or annotate that in `MEMORY.md` (mark something as outdated or corrected). This continuous learning aspect was emphasized: if a hypothesis or prior assumption is corrected, note it in memory as a lesson learned ⁶⁵.

- **Using MEMORY.md:** On new sessions or questions, the agent can quickly scan MEMORY.md (or search within it) to see if the answer is already known. This is a complement to the vector search. Memory.md is human-curated knowledge, which might be easier to scan for straightforward facts. We will make it part of the agent's startup routine to read MEMORY.md (and recent daily memory) as suggested in AGENTS.md ⁶⁹, so it always has that context loaded. This ensures continuity and that the agent doesn't "forget" important insights that were painstakingly discovered.

Overall, MEMORY.md will be managed in a disciplined way: only high-quality, source-backed information goes in, and it's updated at the end of deep research as a final step (the design explicitly calls for updating MEMORY.md with key insights in the post-processing phase) ⁷⁰. This respects Kagakusha's value that "memory is curated wisdom" and not just a pastebin of data. By doing this, future research sessions become more efficient (since Kagakusha can recall prior conclusions without redoing the work) and it provides the **Shachō** with a growing knowledge base.

Invoking Instant vs Deep Modes

The system will support both **instant** and **deep** research workflows, with a clear mechanism to invoke each:

- **Automatic Mode Selection:** By default, the Kagakusha coordinator will attempt to decide the mode based on the query complexity and user input. As mentioned, simple questions trigger the quick summary pathway, while complex ones trigger the deep dive ⁷ ⁸. This automatic detection uses heuristics (keyword detection like "quick overview" vs "in-depth", checking if we have existing info, etc.). The agent's decision logic (encoded in its instructions) will handle this transparently, so the user can just ask a question and get either a short or comprehensive answer as appropriate.
- **User-Directed Invocation:** We will also allow the user (Shachō) to explicitly request a mode. For instance, the user might say, "Give me a quick summary on XYZ" or conversely "I need a comprehensive report on XYZ." The agent will parse that and honor the request. Additionally, if we implement the skills as user-invocable, the user could call something like a command to trigger deep research. According to the design, the `deep-research-instant` skill is marked user-invocable ⁷¹, meaning the user could explicitly invoke the one-pager. Similarly, we could provide a `deep-research` skill for the full report (or use a parameter or follow-up confirmation). In practice, a straightforward way is: after an instant summary is delivered, if the user wants more, they can just say so or use a command. The instant report itself will include a note "(Request deep research mode for more)" ²⁶, prompting the user. If the user confirms, the agent will proceed with the deep research workflow.
- **Session Flow:** In implementation, we might handle these modes within one agent or as separate skills. A simple design: **single agent (Kagakusha)** that can do both. It decides on the fly, but also listens for a user's choice. Alternatively, we could treat instant and deep as two different skills behind the scenes – but given that the orchestrator agent anyway handles planning, it might complicate things to split it. We likely stick to a single orchestrator agent. The conversation might look like:
User: "Research AI agents market in 2025."
Agent: (*decides this needs deep research due to "market in 2025" sounding complex*) – The agent might respond first with a couple **Sōdan (consult)** clarifying questions (e.g., "Do you want a quick overview or a detailed report? Any specific focus, Shachō?") ²⁷. If user says "Just a summary", agent will do instant mode. If user says "Detailed", or if user skipped and just expects detailed, agent will go deep.

In another scenario, the agent might produce an instant summary right away (if it thinks that suffices) and then await if the user wants more.

- **Instant Mode Execution:** When invoked (automatically or by user), Kagakusha will execute the steps of the instant research skill. It will search a few sources, gather facts, and compile the one-page answer with citations. It then outputs this to the chat (and saves it to file). This mode is relatively fast (a single-agent operation, maybe a minute or two). The user can use this as a quick answer. If it fully answers the question, great. If not, the user can escalate to deep mode.
- **Deep Mode Execution:** When deep mode is invoked, Kagakusha will inform the user that it's starting an in-depth investigation (possibly an update like "Renraku: Initiating comprehensive research, this may take a few minutes..." to manage expectations). It will then run through the planning, subagent spawning, and synthesis phases described earlier. Throughout, it might give brief **Renraku (update)** messages to keep the user posted (e.g. "Searching for background info...", "Collecting expert opinions...") ⁷², since deep research could be lengthy. Once completed, the agent compiles the `deep_report.md` and delivers a summary of findings or the full report content (if the platform allows large output) to the user. Often, we might just send the key findings due to length, and mention that the full report is saved to file for reference.
- **Parallel vs Instant Sessions:** OpenCLAW's `sessions_spawn` mechanism means the subagents operate asynchronously in the background. The coordinator can continue to interact in the main chat if needed. However, we will likely design it such that the coordinator waits for all subagents to finish before giving the final answer. The subagents might post intermediate results or a completion message in the chat (by default they "announce results back to the requester channel" on completion) ⁷³. We might suppress overly verbose outputs from subagents by adjusting their prompts (perhaps instruct them to write results to files and not directly to chat unless needed). The coordinator will collate everything and then present the final **Hōkoku (report)**.

In terms of usage, the implementation will make it easy to invoke either mode. We will document to the user (Shachō) that they can ask for a quick summary or a deep report. Internally, we ensure the agent's decision logic and skill triggers cover both paths. This dual-mode setup satisfies the requirement for a "two-tier output system" ⁶.

Finally, whether instant or deep, the **quality standards remain the same**: factual accuracy, proper citations for every fact, and clarity in communication. Kagakusha will deliver either a brief or extensive Kagakusha Report with the same rigor. The flexibility in modes just ensures efficiency – quick answers when that's enough, and heavy research only when truly needed, saving time and API costs.

By following this implementation plan step-by-step, we will build a streamlined, cost-efficient Deep Research Agent that stays true to Kagakusha's identity and the OpenCLAW framework. The agent will operate with the disciplined **Genchi Genbutsu** (go-and-see) research methodology and **Hōrenso** communication protocol, ensuring the Shachō receives high-quality insights with minimal waste. Each component – from subagents to vector memory – has been chosen for simplicity and effectiveness, so the final system should be both powerful and maintainable within ~8–10 pages of code/configuration. With this plan, we can confidently proceed to implement the OpenCLAW Deep Research Agent, knowing it will behave

as a tireless scientist sidekick: eliminating noise, extracting truth. (「Zatsuon haijo. Shinjitsu chūshutsu.」 〇〇)

74

1 2 4 72 74 IDENTITY.md

file:///file_00000000b11071fa929c9f1f1f7c3736

3 5 USER.md

file:///file_0000000009d871fabf1cdea145d39b23

6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35

36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 66 68

70 71 73 deep-research-design.md

file:///file_00000000918871faa5742a7cea648e29

64 65 67 69 AGENTS.md

file:///file_0000000073c871fa829467c6cbf06b83